12A
Metaprogramming in Prolog

Mirko Viroli
mirko.viroli@unibo.it

C.D.L. Magistrale in Ingegneria e Scienze Informatiche
Alma Mater Studiorum—Università di Bologna, Cesena

a.a. 2025/2026

Outline

Previous week:

- The logic programming (and Prolog) paradigm
- Computational model (resolution + unification)
- Basic, idiomatic programming

Previous week:

- some more advanced programming with Prolog
- other ADTs and libs

This week:

- A: metaprogramming
- B: Java/Scala integration with Prolog, by tuProlog

Outline

1. Intro
2. Everything is a term
3. Predicates for solving
4. Dynamic theories
5. Metainterpretation

Metaprogramming

Definition

- "Metaprogramming (or meta-programming) is a programming technique in which computer programs have the ability to treat other programs as their data"
- Accordingly it is possible to:
  - read and reason about programs
  - update/synthesise programs
  - execute programs, possibly in different conditions

The case of Java/Scala

- Java Reflection is a way of inspecting certain parts of a program, even at run-time (other libraries allow for so-called "code instrumentation")
- Scala has advanced techniques to manipulate/execute Scala programs

`https://docs.scala-lang.org/scala3/reference/metaprogramming/index.html`

Metaprogramming in Prolog

Very powerful, simple and direct

- Clauses of a program are actually terms, even syntactically, and this paves the way for virtually supporting any metaprogramming technique
- especially, this is used in AI to tweak Prolog into a component of symbolic reasoning

Ingredients/outline

- "everything is a term"
- libraries to "call" terms
- dynamic theories
- metainterpretation

Experiments in tuProlog

- should better use from now tuProlog 3.3

Outline

1. Intro
2. Everything is a term
3. Predicates for solving
4. Dynamic theories
5. Metainterpretation

Everything is a term

Surface syntax that actually denote standard terms

1. Goals have the same syntax of terms
2. Binary infix operators, e.g.: `X = 1` denotes `'='(X,1)`
   - `+,-,*,/,:-,==,;` ... (there are dozens)
3. Unary prefix operators, e.g.: `-X` denotes `'-'(X)`
   - `+,-,$` ...
4. List syntax:
   - `[A,B,C]` is `'.'(A,'.'(B, '.'(C,'[]')))`
   - `[A,B,C|T]` is `'.'(A,'.'(B, '.'(C,T)))`
5. Comma syntax (when used without functor/predicate):
   - `(A,B,C,D)` is `','(A,','(B,','(C,D)))`
   - `(A)` is `A`
6. Brackets:
   - `{A,B,C,D}` is `'{}'(','(A,','(B,','(C,D))))`
   - `{p(X)}` is `'{}'(p(X))`

```
?- {0,1,2+3} = '{}'(A), A =.. L.
A / (0,1,'+'(2,3))
L / [',',0,(1,'+'(2,3))]
```

Operators

Operators (apparently works in tuProlog 3.3, and have bugs in tuProlog 4.*)

1. an operator is a predicate/functor that is declared to have different syntax, that is, infix if 2-ary or prefix if 1-ary
2. it can also be a symbol without need of using quotes
3. it should be defined by a predicate op/3 to be executed before it is used
   - first argument is a numeric, describing the priority w.r.t. other operators
   - second argument is a literal describing syntax (`xfy`, `yfx`, `xfx`, `fx`)
   - third argument is the operator

```
% use as predicate
:- op(100,xfx,'~').

N1 ~ N2 :- N1 > N2, !, Delta is N1 - N2, Delta < 0.1.
N1 ~ N2 :- N2 ~ N1.
% ?- 10 ~ 10.01. -> Yes

% use as functor
:- op(100,xfy,'::').

to_list(A :: B, [A | B2]) :- to_list(B, B2).
to_list(nil, []).
% ?- to_list(10 :: 20 :: 30 :: nil, L). -> L/[10,20,30]
```

Prolog as host for DSL

```
% use as DSL (IDE 3.3)
:- op(100,fx,[person,name,age,nationality,married]).

person {
    name 'Rossi Marco',
    age 30,
    nationality italy,
    married false
}.

% ?- person { name NM, age A, nationality N, married M }.
% -> NM / 'Rossi Marco'  A / 30  N / italy  M / false
```

Outline

1. Intro
2. Everything is a term
3. Predicates for solving
4. Dynamic theories
5. Metainterpretation

Solving goals

Basic predicates

- `call(+G)`: solves the goal
- `once(+G)`: solves the goal with at most one solution
- `not(+G)`: solves the goal and gives opposite result, without binding
- `true`: always succeeds
- `fail`: always fails

⇒ note that `G` above is actually a term, with all possible syntax

⇒ if `G` is `(G1,...,Gn)` it is handled as a resolvent

Example application:

- "higher-order" predicates, achieved by passing a predicate name

```
1 % map(+L, +P, L0)
2
3 map([], _, []).
4 map([H|T], P, [H2|T2]):- G =.. [P,H,H2], once(G), map(T, P, T2).
5
6 inc(N, N2) :- N2 is N+1.
7
8 % ?- map([10,20,30], inc, L). -> L / [11,21,31]
```

Gathering solutions in lists: `findall/3`

Basic predicates

- `findall(+Res, +Goal, -List)`: solves the goal many times, gather results in list
- `bagof/setof`: variations, less used

Example application

- often simplifies definition of certain algorithms
- sometimes need to get all results upfront, to have a broad view on them
- note the use of `X` and `P` below, as sort of "lambda"

```
1 % ?- findall([X,Y], (member(X,[1,2,3]),member(Y,[1,2,3])), L).
2 % L/[[1,1],[1,2],...,[3,2],[3,3]]
3
4 % filter(+L, X, P, LO)
5 filter(L,X,P,LO) :- findall(X, (member(X,L), once(P)), LO).
6 % ?- filter([10,21,30,40,50], X, X>25, LO).
7 % LO / [30,40,50]
```

Operator ";"

Disjunction of two goals

- at any point of a rule's body, you can indicate a disjunction between two goals (or resolvents)
- `H :- G1, G2, ..., Gk, (G;G'), G1',...,Gk'`
- informal meaning: "either `G` or `G'` (or both)" should be solvable
- "formal" semantics: extract two new clauses `C(X1,..,Xn) :- G` and `C(X1,..,Xn) :- G'` where `X1,..,Xn` are all the variables used in `G` and `G'`, and replace `(G;G')` with `C(X1,..,Xn)`
- hence, note that ";" may introduce a branch

```
1 neighbour(X,Y1,X,Y2) :- Y1 is Y2+1 ; Y1 is Y2-1
2 neighbour(X1,Y,X2,Y) :- X1 is X2+1 ; X1 is X2-1
```

Other control predicates

- `->`, `repeat`, ...
- they are a step too far, you wouldn't use them: if you use them you will be asked what is their exact semantics

Outline

1. Intro
2. Everything is a term
3. Predicates for solving
4. Dynamic theories
5. Metainterpretation

Changing programs

The dynamic theory

- a Prolog program is actually made of 2 "theories" (sets of predicates/clauses)
  - the static one, typically provided at the beginning of the "computations session": the one written in the IDE
  - a dynamic one, initially empty, where you can add/remove clauses dynamically – clauses are terms, hence the API is straightforward

  $\Rightarrow$ some Prologs (e.g. tuProlog 3.3 or 4.0) have a single theory, and it is dynamic
- not surprisingly, a Prolog theory can be inspected programmatically...

Predicates

- `assert(+Clause)`, adds a clause, always succeeds, never binds
  - e.g.: `assert(p(1))`, `assert(p(X):-q(X))`.
  - `asserta`/`assertz` assert on top/bottom
- `retract(+Clause)`, retract a matching clause if exists, and possibly binds
- `retractall(+Clause)`, retract all matching clauses, always succeeds, never binds
- `clause(?Head,?Body)`, queries for a clause (possibly with many solutions and binds)
  - `Body` gives `true` if empty, or `(G1,...,Gn)` in general

Example: caching factorial in the theory

```
factorial(0,1).
factorial(X,Y):- Xm is X-1, factorial(Xm,Y2), Y is Y2*X.

withcache(P) :- cached(P), !.
withcache(P) :- once(P), assert(cached(P)).
```

```
?- cached(X). -> No
?- factorial(3,N). -> N/6
?- cached(X). -> No
?- withcache(factorial(3,N)). -> N/6
?- cached(X). -> X/factorial(3,6)
?- clause(cached(X),B). -> X/factorial(3,6), B/true
?- withcache(factorial(4,N)). -> N/24
?- findall(C, cached(C), L). -> L/[factorial(3,6),factorial(4,24)]
```

Note

- will see that `withcache` can be understood as a metainterpreter for single-result predicates, adding caching to standard interpretation

Outline

1. Intro
2. Everything is a term
3. Predicates for solving
4. Dynamic theories
5. Metainterpretation

Metainterpretation

Interpreter

An interpreter is a program that reads and executes another program.

Metainterpreter

A metainterpreter is an interpreter written in the same language of the program to be interpreted – typically, you do so to be able to enact some change on the interpretation strategy

Metainterpretation in Prolog

- Actually somewhat easy, since programs are just terms (data), and it is easy to perform "resolution"-like mechanisms to substitute heads with bodies

Applications

- for Prolog programmers: debugging, profiling, changing resolution strategy
- in AI planning and similar: to perform ad-hoc "space-exploration" on your DSL

Preliminary step: simulating resolution

```
rule(a, []).    % means:    a.
rule(b, []).    % means:    b.
rule(d, []).    % means:    d.
rule(d, []).    % means:    d.
rule(c, []).    % means:    c.
rule(c, [c]).   % means:    c :- c.

% we are using unification when calling rule/2
% we branch because calling rule/2 branches
% append performs head/body susbtitution
solve([]).
solve([Goal | Rest]) :-
    rule(Goal, Body), append(Body, Rest, NewGoals), solve(NewGoals).
```

```
?- solve([a]). -> yes
?- solve([d]). -> yes; yes
?- solve([e]). -> no
?- solve([c]). -> yes; yes; yes; ...
```

Preliminary step: it is actual "full resolution"!

```
rule(a, []).    % means:    a.
rule(b, []).    % means:    b.
rule(d, []).    % means:    d.
rule(d, []).    % means:    d.
rule(c, []).    % means:    c.
rule(c, [c]).   % means:    c :- c.

% we are using unification when calling rule/2
% we branch because calling rule/2 branches
% append performs head/body susbtitution
solve([]).
solve([Goal | Rest]) :-
    rule(Goal, Body), append(Body, Rest, NewGoals), solve(NewGoals).

rule(search(E,[E|_]), []).
rule(search(E,[_|T]), [search(E,T)]).
```

```
?- solve([a]). -> yes
?- solve([d]). -> yes; yes
?- solve([e]). -> no
?- solve([c]). -> yes; yes; yes; ...

?- solve([search(X,[10,20,30])]). -> X/10; X/20; X/30; % !!!!
% it works! we just now need to digest standard Prolog clauses...
```

The Vanilla metainterpreter

```
1 solveV(true) :- !.
2 solveV((A,B)) :- !, solveV(A), solveV(B).
3 solveV(G) :- clause(G, B), solveV(B).
```

```
1 search(E,[E|_]).
2 search(E,[_|T]) :- search(E,T).
3
4 ?- solveV(search(X,[10,20,30])). -> X/10; X/20; X/30; % !!!!
```

Defects. . . or features?

- It cannot deal with built-in library predicates (not defined by clauses)
- Control predicates must be "reimplemented" in the metainterpreter, if needed

$\Rightarrow$ . . . this might be considered a feature, since it allows you to pick what to use explicitly

Metainterpretation with built-ins and control

```
builtins(['is', '=','\\=','==']). % possibly more
solveB(true) :- !.
solveB((A,B)):- !, solveB(A), solveB(B).
solveB(G):- functor(G, F, _), builtins(B), member(F,B), !, call(G).
solveB(once(G)) :- !, once(solveB(G)). % handling once/1
solveB(not(G)) :- !, not(solveB(G)).   % handling not/1
solveB(G) :- clause(G,B), solveB(B).
```

```
sum([], 0).
sum([H|T], N) :- sum(T,N2), N is N2+H.

?- solveB( sum([10,20,30], S) ). -> Yes S/60
?- solveB( not(sum([10,20,30], 50)) ). -> Yes
?- solveB( not(sum([10,20,30], 60)) ). -> No
```

Notes

- Rule 4 deals with built-ins, to be called as such (`call(G)`)
- Rules 5, 6 deal with one control predicate at a time
- Challenge for you: how to implement cut??

Metainterpretation altering order of solutions

```
solveI(true):- !.
solveI((A,B)):- !, solveI(A), solveI(B).
solveI(G) :-
  findall(c(G, B), clause(G, B), L),
  reverse(L, L2),  % inverting solutions!!
  member(c(G, B), L2),
  solveI(B).
```

```
?- solveI(search(X,[10,20,30])). -> X/30; X/20; X/10; % !!!!
```

Notes

- this general approach can be used to collect "branches", analyse them, and decide what to prioritize and/or discard

Metainterpretation to track and constrain computations

```
% solveT(+Goal, -Trace, -SizeBound)
solveT(G, T, B) :- solveT(G, T, B, _).
solveT(true, [], N, N) :- !.
solveT((A,B), L, IN, ON) :- !,
  solveT(A, LA, IN, ON1),
  findall( (X,B), member(X,LA), LL),
  solveT(B, LB, ON1, ON),
  append(LL, LB, L).
solveT(G, [G|T], I, O) :-
    I > 0, !, clause(G,B), I2 is I-1, solveT(B,T, I2, O).
```

```
?- solveT(search(X,[10,20,30]), T, 2).
-> X/10, T/[search(10,[10,20,30])]
-> X/20, T/[search(20,[10,20,30]),search(20,[20,30])]
no
```

Notes

- this general approach can be used to provide solution along with whole computation trace, useful for debugging purposes or for constraining computations – e.g. pruning long computations